MLP và backpropagation

Tự xây mạng neural và lan truyền ngược.

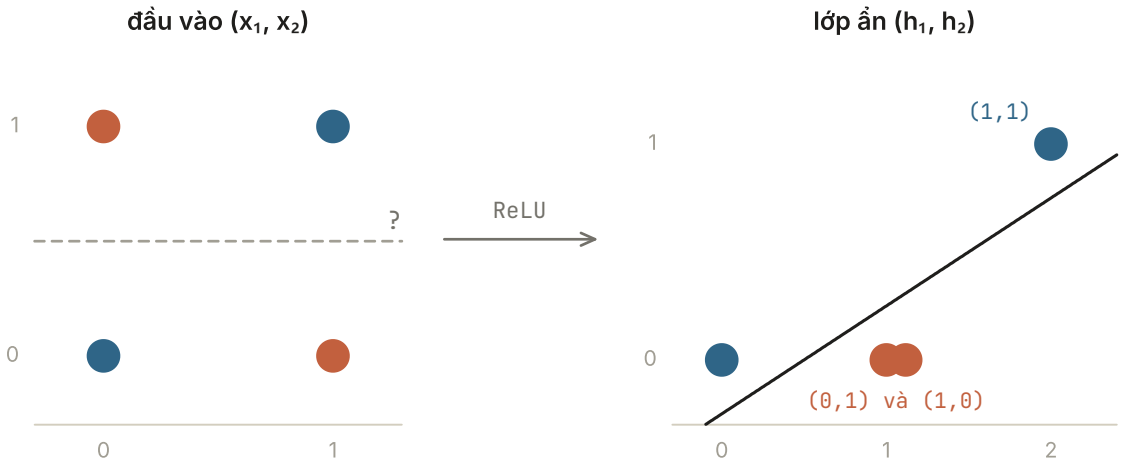
> Hai công tắc điều khiển một đèn: đèn sáng khi đúng một công tắc bật. Quy tắc XOR rất quen trong CP, nhưng một mô hình chỉ cộng trọng số đầu vào không thể chia bốn trường hợp bằng một đường thẳng. Ta cần tạo đặc trưng trung gian rồi kết hợp chúng. MLP (multilayer perceptron) thực hiện điều đó bằng các lớp linear xen activation. Câu hỏi xuyên suốt là: làm sao lỗi ở đầu ra biết phải quay về chỉnh trọng số ở lớp đầu?

MỤC TIÊU SAU BÀI NÀY

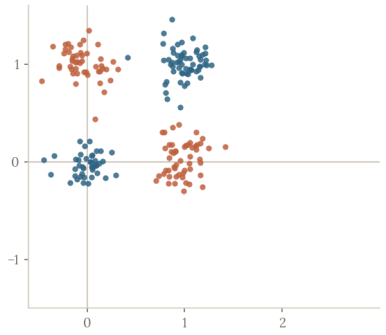
- Giải thích vai trò của hidden layer và activation qua XOR.
- Ghi shape của forward và backward cho mạng hai lớp.
- Tính một lượt backprop bằng tay, cài bằng NumPy và đối chiếu PyTorch.
- Phân biệt kiểm gradient, kiểm khả năng học và đánh giá khả năng tổng quát hóa.

Vì sao thêm một lớp?

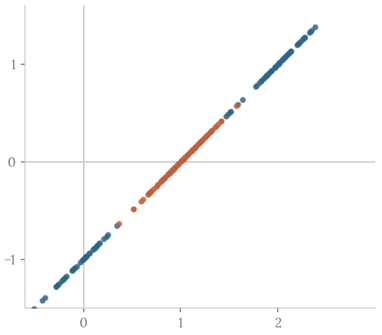
Với đầu vào x_1, x_2 thuộc $\{0, 1\}$, XOR có nhãn 1 tại $(0, 1), (1, 0)$ và nhãn 0 ở hai góc còn lại. Một đường thẳng không tách được hai cặp góc đối diện. Hình dưới giúp thấy hạn chế về biểu diễn trước khi bàn tới thuật toán tối ưu: training lâu hơn không chữa được kiến trúc không biểu diễn nổi hàm cần học.



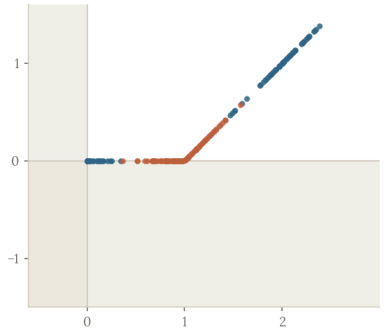
Trong mặt phẳng đầu vào, không đường thẳng nào tách hai cặp góc XOR; qua lớp ẩn ReLU, hai góc nhãn 1 dồn về (1, 0) và một đường thẳng tách được.



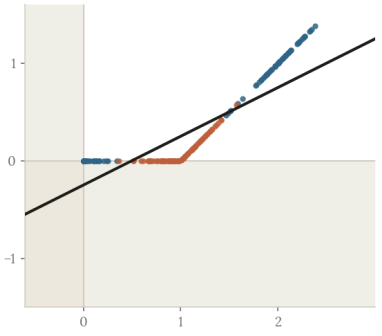
đầu vào (x_1, x_2): không đường thẳng nào tách hai cặp góc



z = xW + b với W toàn 1, b = (0, −1): $z_2 = z_1 - 1$, mọi điểm dồn lên một đường thẳng



ReLU: tọa độ âm bị ép về 0, nửa dưới gập lên trục ngang



$h_1 - 2h_2 = 0.5$ tách hai lớp: đúng 98% trong 200 điểm

Lớp ẩn đặt tay của bài biến 200 điểm XOR: phép tuyến tính dồn mọi điểm lên một đường thẳng, ReLU gập phần âm về trục, rồi một đường thẳng tách được hai lớp.

Tạo hai đặc trưng $h_1 = \text{ReLU}(x_1 + x_2)$ và $h_2 = \text{ReLU}(x_1 + x_2 - 1)$, với $\text{ReLU}(z) = \max(0, z)$. Khi tổng là 0, 1, 2, hai đặc trưng lần lượt là $(0, 0)$, $(1, 0)$, $(2, 1)$. Công thức $\hat{y} = h_1 - 2h_2$ cho 0, 1, 0, đúng XOR. Đây là một mạng được **đặt trọng số bằng tay**, chưa phải mạng đã học.

Python

```python
import numpy as np
import torch

torch.set_num_threads(1)
torch.manual_seed(11)
np.set_printoptions(precision=4, suppress=True)
X_xor = np.array([[0., 0.], [0., 1.], [1., 0.], [1., 1.]])
hidden = np.maximum(X_xor @ np.ones((2, 2)) + [0., -1.], 0.)
print(hidden)
print(hidden @ np.array([1., -2.]))
```

KẾT QUẢ

```
[[0. 0.]
 [1. 0.]
 [1. 0.]
 [2. 1.]]
[0. 1. 1. 0.]
```

Dòng cuối cần cho `0, 1, 1, 0`. Hidden layer đã biến bốn góc thành những vị trí mà lớp sau kết hợp được: hai góc $(0,1), (1,0)$ cùng rơi vào điểm $(h_1, h_2) = (1,0)$, còn góc $(1,1)$ bị đẩy ra $(2,1)$.

HỎI NHANH

Giữ nguyên hai đặc trưng h_1, h_2 của mạng XOR đặt tay, chỉ đổi đầu ra thành $\hat{y} = h_1 - h_2$. Với bốn đầu vào $(0,0)$, $(0,1)$, $(1,0)$, $(1,1)$, mạng cho ra hàm nào?

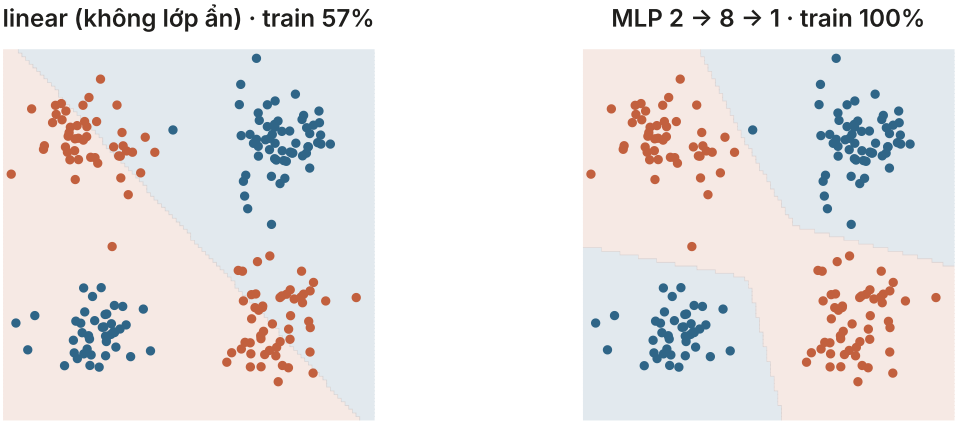
A OR: $0, 1, 1, 1$

B AND: $0, 0, 0, 1$

C Vẫn là XOR: $0, 1, 1, 0$

D NAND: $1, 1, 1, 0$

Nếu bỏ activation, hai lớp gộp lại: $(XW_1 + b_1)W_2 + b_2 = X(W_1W_2) + (b_1W_2 + b_2)$, vẫn là một lớp linear. Thêm tham số mà không thêm phi tuyến không giải quyết được XOR.



200 điểm quanh bốn góc XOR, cùng 500 bước Adam: mô hình linear chỉ đúng 57%; thêm một lớp ẩn 8 neuron ReLU thì học được cả bốn vùng.

HỎI NHANH

Mạng $2 \to 100 \to 1$ gồm hai lớp linear nối thẳng, không có activation ở giữa. Mạng này biểu diễn được những hàm nào?

A Mọi hàm liên tục, nhờ 100 neuron ẩn

B Đúng những hàm mà một lớp linear biểu diễn được

C Thêm được XOR, nhờ lớp ẩn tạo ra đặc trưng mới

D Nhiều hơn một lớp linear, nhưng chưa tới XOR

Forward có bảng shape

Ta thống nhất mẫu nằm ở hàng. Một batch X có shape `(B, d)`, hidden width h, đầu ra c:

$$Z_1 = XW_1 + b_1, \qquad H = \mathrm{ReLU}(Z_1), \qquad P = HW_2 + b_2.$$

Tên	Shape	Ý nghĩa
W_1, b_1	`(d, h), (h,)`	Tạo hidden feature
Z_1, H	`(B, h)`	Trước và sau activation

Tên	Shape	Ý nghĩa
W_2, b_2	`(h, c)`, `(c,)`	Tạo đầu ra
P, Y	`(B, c)`	Dự đoán và target

Trong NumPy ta dùng cách viết hàng như bảng này. `torch.nn.Linear(d,h)` lưu `weight` shape `(h,d)` và thực hiện $X\,\text{weight}^\top + b$. Hai quy ước cùng đúng; khi chuyển trọng số tự viết sang `Linear`, cần chuyển vị. Nếu copy thẳng ma trận vuông thì shape vẫn hợp lệ nhưng nội dung có thể sai.

Forward phải lưu X, Z_1, H cho backward, vì đạo hàm phụ thuộc chúng. Đó là nguồn chi phí activation memory: nhớ giá trị trung gian để khỏi tính lại. Backprop là thuật toán đi ngược các phép, còn autograd là cơ chế thư viện xây graph và gọi đạo hàm phù hợp.

HỎI NHANH

Lớp đầu đã được cài bằng NumPy theo quy ước hàng, $Z = XW_1$, với `W1` có shape `(3, 3)`. Chép thẳng `W1` vào `weight` của `torch.nn.Linear(3, 3)` rồi chạy. Chuyện gì xảy ra?

A Báo lỗi shape ngay ở lần forward đầu tiên

B Ra đúng kết quả, vì PyTorch tự chuyển vị khi cần

C Chạy êm, nhưng tính ra $XW_1^\top$, sai mà không báo

D Chỉ bias bị sai, phần trọng số vẫn đúng

Từ loss về lớp đầu

Để dễ kiểm, ta dùng hồi quy một đầu ra $c = 1$ và $J = \frac{1}{B}\sum_i (P_i - Y_i)^2$. Bài 3.3 sẽ đổi sang loss phân loại. Đặt $G_2 = \partial J/\partial P = 2(P - Y)/B$, shape `(B,1)`. Với lớp linear cuối:

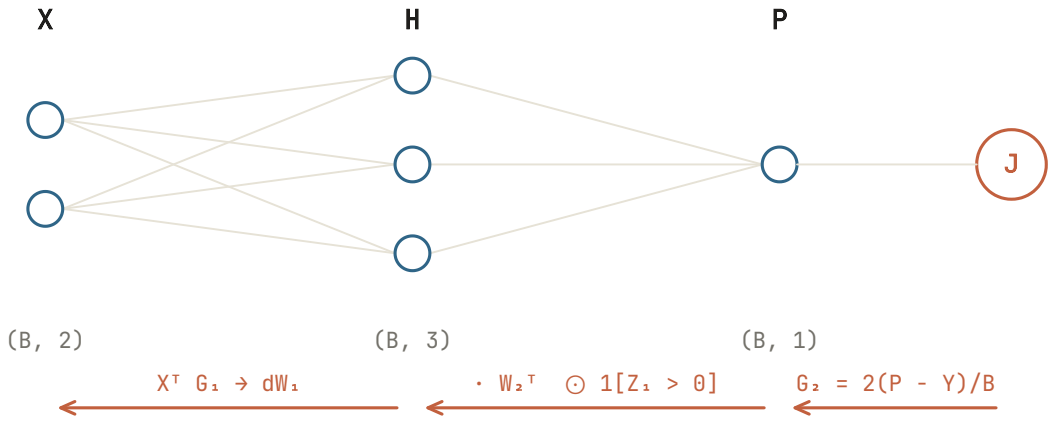
$$\frac{\partial J}{\partial W_2} = H^\top G_2, \qquad \frac{\partial J}{\partial b_2} = \sum_i G_{2,i}.$$

Đạo hàm theo hidden là $G_2 W_2^\top$. Khi có nhiều đầu ra, mỗi hidden neuron nối tới tất cả nên ảnh hưởng tới loss qua nhiều đường; chain rule cộng đóng góp của các đường ấy, và phép nhân ma trận làm sẵn phép cộng này. Qua ReLU, phần có $Z_1 < 0$ không đổi đầu ra khi xê dịch nhỏ nên gradient bằng 0; phần $Z_1 > 0$ đi qua nguyên vẹn:

$$G_1 = (G_2 W_2^\top) \odot \mathbf{1}[Z_1 > 0], \qquad \frac{\partial J}{\partial W_1} = X^\top G_1, \qquad \frac{\partial J}{\partial b_1} = \sum_i G_{1,i}.$$

$\odot$ là nhân từng phần tử, không phải nhân ma trận. Tại đúng 0, ReLU không có đạo hàm cổ điển; ta và PyTorch chọn gradient 0. Khi kiểm bằng sai phân, tránh đặt điểm đúng chỗ gãy ấy. Nếu loss lấy trung bình trên nhiều đầu ra c, hệ số sẽ là $2/(Bc)$; công thức ở đây dùng $c = 1$.

Hình dưới vẽ đường đi ấy cho mạng $2 \to 3 \to 1$ sẽ dựng bằng PyTorch ở mục dùng thư viện: gradient từ J về P, nhân $W_2^\top$ để về H, qua cổng ReLU′ rồi mới tới lớp đầu.



Mạng 2 → 3 → 1: forward đi từ trái sang; gradient đi ngược từ J, nhân $W_2^\top$ để về lớp ẩn, qua cổng ReLU′ rồi mới tới W_1.

HỎI NHANH

`H` có shape `(5, 4)`, `G2` có shape `(5, 1)`. Gradient theo `W2` có shape gì?

A `(5, 5)`

B `(5, 4)`

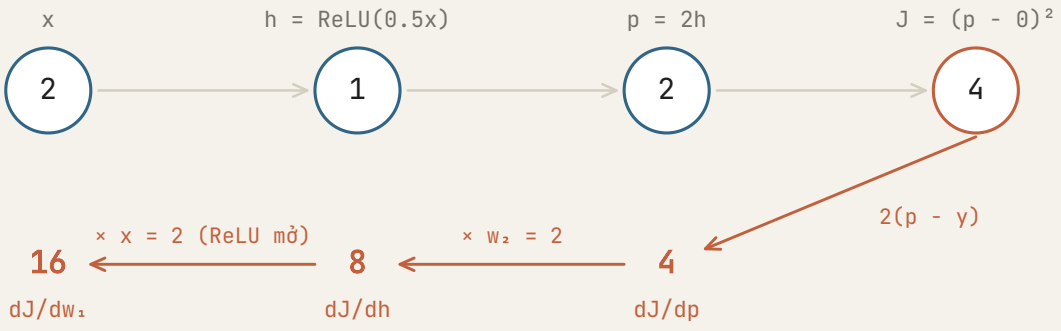
C `(1, 4)`

D `(4, 1)`

Ví dụ tính tay

Lấy một mẫu $x = 2, y = 0$, mạng một hidden neuron: $w_1 = 0.5, b_1 = 0, w_2 = 2, b_2 = 0$. Forward có $z_1 = 1, h = 1, p = 2, J = 4$. Bắt đầu từ đầu ra: $dJ/dp = 4$, nên $dJ/dw_2 = 4h = 4$ và $dJ/db_2 = 4$. Truyền về hidden cho $dJ/dh = 4w_2 = 8$; ReLU đang mở vì $z_1 > 0$, nên $dJ/dw_1 = 8x = 16$, $dJ/db_1 = 8$.

Các gradient dùng **trọng số trước cập nhật**. Nếu sửa w_2 rồi mới tính gradient lớp đầu, đang trộn hai mạng khác nhau trong cùng lượt backward. Tính đủ gradient trước, cập nhật tất cả sau. Khi một neuron âm ở mọi mẫu, gradient qua ReLU của nó bằng 0; vấn đề này sẽ nối sang activation ở bài tiếp.



Ví dụ tính tay x = 2, y = 0: forward 2 → 1 → 2 → 4; backward nhân dần 4 → 8 → 16 để ra $dJ/dw_1 = 16$.

HỎI NHANH

Mạng một mẫu có $h = \text{ReLU}(w_1 x)$, $p = w_2 h$, $L = (p - y)^2$. Tại $x = 3$, $w_1 = 1$, $w_2 = 1$, $y = 1$, gradient $\partial L / \partial w_1$ bằng bao nhiêu?

A 6

B 3

C 12

D 4

Cài nhỏ bằng NumPy

Ta kiểm trên hai mẫu, hai hidden neuron và điểm không nằm ở chỗ gãy. Tách forward/backward thành hàm để cùng một code phục vụ tính gradient và thử loss sau cập nhật:

Python

```
X = np.array([[1., 2.], [2., -1.]])
Y = np.array([[1.], [0.]])
params = [np.array([[.2, -.3], [.4, .1]]), np.array([.1, .2]),
          np.array([[.5], [-.4]]), np.array([.05])]

def forward(params):
    W1, b1, W2, b2 = params
    Z1 = X @ W1 + b1
    H = np.maximum(Z1, 0)
    P = H @ W2 + b2
    return P, (Z1, H)

def objective(params):
    P, _ = forward(params)
    assert P.shape == Y.shape
    return np.mean((P - Y) ** 2)
```

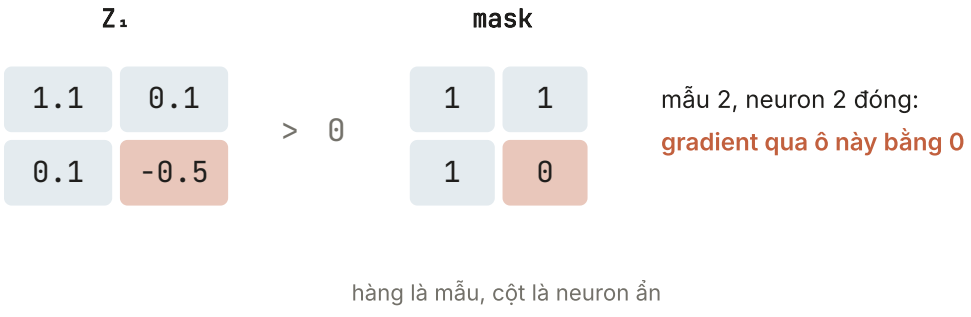
```python
def backward(params):
    P, (Z1, H) = forward(params)
    G2 = 2 * (P - Y) / len(X)
    G1 = (G2 @ params[2].T) * (Z1 > 0)
    return [X.T @ G1, G1.sum(axis=0), H.T @ G2, G2.sum(axis=0)]

print(forward(params)[0].ravel())
print(round(objective(params), 6))
for g in backward(params):
    print(g)
```

KẾT QUẢ

```
[0.56 0.1 ]
0.1018
[[-0.12   0.176]
 [-0.49   0.352]]
[-0.17   0.176]
[[-0.474]
 [-0.044]]
[-0.34]
```

Hai dự đoán là 0.56 và 0.10, nên loss tính tay $(0.44^2 + 0.10^2)/2 = 0.1018$. Kiểm từng ma trận gradient cùng shape với trọng số tương ứng. Chẳng hạn dJ/dW_2 hàng đầu bằng $1.1(-0.44) + 0.1(0.10) = -0.474$; đây là tổng hai đường đóng góp qua hai mẫu. Ở batch này $Z_1 = \begin{bmatrix} 1.1 & 0.1 \\ 0.1 & -0.5 \end{bmatrix}$: neuron ẩn thứ hai đóng ở mẫu thứ hai, và mask `(Z1 > 0)` chặn gradient qua đúng ô đó.



Ở batch hai mẫu của bài, Z_1 chỉ có một ô âm; mask $(Z_1 > 0)$ chặn gradient qua đúng ô đó, ba ô còn lại cho gradient đi qua.

HỎI NHANH

Hàm `backward` ở trên bị quên `* (Z1 > 0)`. Với Z_1 của batch này, những gradient nào bị sai?

A Cả bốn gradient, vì G_1 có mặt trong mọi công thức

B Chỉ phần W_1, b_1 nối vào neuron ẩn thứ hai

C Chỉ W_2 và b_2, vì chúng nằm sát ReLU nhất

D Không gradient nào, vì forward đã cắt H về 0

Đối chiếu độc lập rồi mới học

Sai phân trung tâm ước lượng đạo hàm theo một phần tử tham số θ_k bằng $\big(J(\theta_k + \varepsilon) - J(\theta_k - \varepsilon)\big)/(2\varepsilon)$, giữ nguyên các phần tử khác. Mỗi phần tử tốn hai lần tính loss. Cách này chậm nhưng không dùng chung công thức nào với backward, giống lời giải brute force dùng để kiểm thuật toán tối ưu trong CP. Ta dùng `float64` và $\varepsilon = 10^{-5}$: sai số của sai phân trung tâm cỡ ε^2, sai số làm tròn của `float64` còn nhỏ hơn, nên so với ngưỡng `atol=1e-8` là đủ chặt. Mọi phần tử của Z_1 cách 0 ít nhất 0.1, xa hơn nhiều so với bước xê dịch, nên phép thử không vượt qua chỗ gãy của ReLU. Với `float32`, sai số làm tròn bị khuếch đại khi chia cho 2ε; khi kiểm, chuyển mạng tí hon sang `float64`.

Python

```python
def numerical_gradient(params, eps=1e-5):
    answer = []
    for p in params:
        g = np.zeros_like(p)
        for index in np.ndindex(p.shape):
            old = p[index]
            p[index] = old + eps
            plus = objective(params)
            p[index] = old - eps
            minus = objective(params)
            p[index] = old
```

```
            g[index] = (plus - minus) / (2 * eps)
        answer.append(g)
    return answer

manual = backward(params)
numeric = numerical_gradient(params)
print('numeric match:', all(np.allclose(a, b, atol=1e-8)
                            for a, b in zip(manual, numeric)))

tp = [torch.tensor(p, dtype=torch.float64, requires_grad=True) for p in params]
tx, ty = torch.tensor(X), torch.tensor(Y)
pred = torch.relu(tx @ tp[0] + tp[1]) @ tp[2] + tp[3]
((pred - ty) ** 2).mean().backward()
print('autograd match:', all(np.allclose(g, p.grad.numpy(), atol=1e-10)
                            for g, p in zip(manual, tp)))
updated = [p - .05 * g for p, g in zip(params, manual)]
print(round(objective(params), 6), round(objective(updated), 6))
```

KẾT QUẢ

```
numeric match: True
autograd match: True
0.1018 0.065513
```

Hai dòng `True` nói backward khớp với hai cách tính độc lập tại bộ tham số đã chọn. Dòng cuối cho thấy một bước gradient descent với learning rate 0.05 đưa loss từ 0.1018 xuống 0.065513. Hai loại bằng chứng này không thay được nhau. Loss giảm sau một bước chỉ cần hướng cập nhật đi xuống, nên một backward sai ở vài phần tử vẫn có thể làm loss giảm. Ngược lại, gradient đúng không hứa hội tụ tới nghiệm toàn cục, vì loss của mạng neural thường không lồi.

HỎI NHANH

Mạng nhỏ ở trên có 9 phần tử tham số. Kiểm bằng sai phân trung tâm thì phải tính `objective` bao nhiêu lần? Với một MLP có 10^6 tham số thì sao?

A 18 lần, và mạng 10^6 tham số cần $2 \cdot 10^6$ lần

B 9 lần, và mạng 10^6 tham số cần 10^6 lần

C 2 lần, số tham số không ảnh hưởng chi phí

D 4 lần, mỗi tensor một cặp, và mạng lớn vẫn chỉ 4 lần

HỎI NHANH

Gradient tính bằng NumPy trùng với autograd và với sai phân trên một batch nhỏ. Điều đó bảo đảm được gì?

A Mạng sẽ có điểm validation cao sau khi train

B Gradient descent sẽ tới được nghiệm toàn cục

C Mọi batch khác cũng sẽ cho loss giảm

D Backward tính đúng đạo hàm tại điểm đã kiểm

Dùng thư viện ở đúng tầng

Khi đã hiểu cơ chế, bài thực tế viết `torch.nn.Sequential(torch.nn.Linear(d,h), torch.nn.ReLU(), torch.nn.Linear(h,c))` để thư viện quản lý tham số và autograd lo phần backward. Trước khi train, nên hỏi lại mô hình hai điều: output có shape như dự tính không, và mạng có bao nhiêu tham số:

```
Python
```

```
model = torch.nn.Sequential(torch.nn.Linear(2, 3), torch.nn.ReLU(),
                            torch.nn.Linear(3, 1))
print(tuple(model(torch.zeros(4, 2)).shape))
print(sum(p.numel() for p in model.parameters()))
```

KẾT QUẢ

```
(4, 1)
13
```

Đầu ra `(4,1)` là bốn dự đoán. Số tham số là $2 \cdot 3 + 3 + 3 \cdot 1 + 1 = 13$, tính cả hai bias. Layer ReLU không có tham số học. Đặt đầu ra nhị phân là một logit và dùng BCEWithLogitsLoss, hoặc nhiều lớp loại trừ nhau là c logits và CrossEntropyLoss; không ghép loss chỉ vì tensor cùng shape.

HỎI NHANH

Mạng hồi quy trả về `pred` shape `(4, 1)`, giá trị đúng bằng target `y = torch.tensor([1., 2., 3., 4.])` có shape `(4,)`. Thế nhưng `torch.nn.MSELoss()(pred, y)` lại cho 2.5 thay vì 0. Vì sao?

A `MSELoss` mặc định cộng các bình phương thay vì lấy trung bình

B Sai số làm tròn của `float32` cộng dồn qua bốn mẫu

C `pred - y` bị broadcast thành `(4, 4)`, so mọi cặp mẫu

D `y` phải đổi sang `float64` cho cùng kiểu với `pred`

Bẫy cài đặt và chi phí

Nếu mạng không học nổi vài mẫu, kiểm shape, hệ số trung bình loss, ReLU đóng, gradient bị detach và thứ tự cập nhật. Gradient bias là tổng theo batch, không phải lấy mean thêm lần nữa nếu G_2 đã chứa $1/B$. Khi đổi quy ước ma trận hàng/cột, ghi bảng shape lại từ đầu. Sao chép ma trận vuông mà quên chuyển vị là lỗi dễ sống sót qua kiểm shape.

Học sinh CP đã quen xét độ phức tạp. Với MLP hai lớp, forward chủ yếu tốn $O(Bdh + Bhc)$ phép tính; hidden activation cần khoảng $O(Bh)$ số. Tăng hidden width làm cả chi phí và khả năng biểu diễn tăng, nhưng không bảo đảm validation tốt hơn. Giữ baseline linear, tập validation cố định và seed khi so mạng nhỏ với mạng rộng.

HỎI NHANH

Code backward viết `db2 = G2.mean(axis=0)`, trong khi `G2 = 2 * (P - Y) / len(X)`. Với batch 64 mẫu, gradient bias này sai thế nào?

A Đúng, vì bias dùng chung cho cả batch

B Nhỏ hơn đúng 64 lần, vì đã chia cho B hai lần

C Lớn hơn đúng 64 lần, vì lấy trung bình thay cho tổng

D Sai dấu, vì lấy trung bình làm đổi chiều của tổng

Khi đi thi

- **Học thuộc một batch nhỏ trước.** Cho mạng train trên vài chục mẫu đến khi loss gần 0. Không làm được thì lỗi nằm ở code (shape, mask ReLU, `detach`, thứ tự cập nhật, hệ số trung bình), chưa phải ở hyperparameter.
- **Tự viết backward thì kiểm sai phân trước.** Mạng tí hon, `float64`, điểm xa chỗ gãy của ReLU; khớp rồi mới chạy trên dữ liệu thật.
- **Đếm tham số và bộ nhớ trước khi train.** Một lớp ẩn có $dh + h + hc + c$ tham số và activation cỡ Bh số; lớp nối với đầu vào lớn thường chiếm gần hết tham số.
- **So với baseline linear trên cùng split và seed.** MLP rộng hơn tốn thêm thời gian GPU nhưng không chắc tốt hơn trên validation.
- **Đề yêu cầu đặt trọng số tay** (hàm ngưỡng, không huấn luyện): mỗi neuron ẩn là một nửa mặt phẳng, neuron cuối ghép chúng bằng AND hoặc OR qua ngưỡng; thử lại trên đỉnh, cạnh và vài điểm ngoài.

- **Chép trọng số sang `nn.Linear` thì chuyển vị**, rồi so output hai bản trên vài mẫu.

Tóm tắt

Hidden layer tạo đặc trưng; activation giúp biểu diễn hàm phi tuyến. Backprop đi ngược graph, nhân độ nhạy cục bộ và cộng các nhánh. Gradient có cùng shape với tham số, mọi gradient phải tính trước cập nhật. NumPy, sai phân và autograd cho ba cách kiểm độc lập trên dữ liệu nhỏ; đánh giá validation vẫn là bước khác.

Kiểm tra cuối bài

CÂU 1

Đề yêu cầu dựng một MLP chỉ gồm lớp linear và hàm ngưỡng $\Theta(z) = 1$ nếu $z \geq 0$, ngược lại bằng 0. Trọng số đặt tay, không huấn luyện. Đầu ra phải bằng 1 khi điểm (x, y) nằm trong tam giác có đỉnh $(0, 0)$, $(3, 1)$, $(1, 5)$, tính cả cạnh. Lớp ẩn có $h_1 = \Theta(3y - x)$, $h_2 = \Theta(7 - 2x - y)$, $h_3 = \Theta(5x - y)$. Chọn neuron đầu ra.

A $\Theta(h_1 + h_2 + h_3 - 1)$

B $\Theta(h_1 + h_2 + h_3 - 2)$

C $\Theta(h_1 + h_2 + h_3 - 3)$

D $\Theta(3 - h_1 - h_2 - h_3)$

PHỎNG THEO USA-NA-AIO 2026 · Vòng 1 · Part 7.4

CÂU 2

Mạng không có bias. $z_1 = w_A x_1 + w_B x_2$, $a_1 = g(z_1)$. Từ a_1 tách ra hai nhánh: $z_2 = w_C a_1$, $a_2 = g(z_2)$ và $z_3 = w_D a_1$, $a_3 = g(z_3)$. Cuối cùng $z_4 = w_E a_2 + w_F a_3$, $\hat{y} = g(z_4)$. Đặt $\delta = \partial \ell / \partial z_4$. Biểu thức nào là $\partial \ell / \partial a_1$?

A $\delta \left(w_E g'(z_2) w_C + w_F g'(z_3) w_D \right)$

B $\delta \, w_E g'(z_2) w_C$

C $\delta \left(w_E g'(z_2) w_C \right) \left(w_F g'(z_3) w_D \right)$

D $\delta \left(w_E w_C + w_F w_D \right)$

PHỎNG THEO CMU 10-601 · Exam 2 Practice · Q4.4

CÂU 3

Với `lin = torch.nn.Linear(2, 3)`, `lin.weight` và `lin.bias` có shape gì?

A `(2, 3)` và `(3,)`

B `(2, 3)` và `(2,)`

C `(3, 2)` và `(2,)`

D `(3, 2)` và `(3,)`

CÂU 4

Ảnh xám 28×28 được duỗi thành vector, rồi đưa qua MLP $784 \to 128 \to 10$, cả hai lớp đều có bias. Mạng có bao nhiêu tham số?

A 100480

B 101632

C 101760

D 101770

CÂU 5

Mạng một đầu vào: $a_1 = w_{11}x + b_{11}$, $a_2 = w_{12}x + b_{12}$, $z_j = \mathrm{ReLU}(a_j)$, $a_3 = w_{21}z_1 + w_{22}z_2 + b_{21}$, $\hat{y} = \sigma(a_3)$. Loss là cross-entropy nhị phân, nên $\partial L/\partial a_3 = \hat{y} - y$. Ở một mẫu có $a_1 = 0.5$, $a_2 = -0.2$, với $w_{21} = 1$, $w_{22} = -1$. Gradient theo hai bias của lớp ẩn thế nào?

A $\partial L/\partial b_{11} = 0$ và $\partial L/\partial b_{12} = 0$

B $\partial L/\partial b_{11} \neq 0$ và $\partial L/\partial b_{12} = 0$

C $\partial L/\partial b_{11} = 0$ và $\partial L/\partial b_{12} \neq 0$

D $\partial L/\partial b_{11} \neq 0$ và $\partial L/\partial b_{12} \neq 0$

PHỎNG THEO CMU 10-601 · Exam 2 Practice · Q4.2(d)

CÂU 6

Mạng một lớp ẩn: $z_1 = w_{11}x_1 + w_{12}x_2$, $z_2 = w_{21}x_1 + w_{22}x_2$, $a_j = f(z_j)$ với f là sigmoid. Đầu ra trước activation là $z = v_1 a_1 + v_2 a_2$. Loss được cộng thêm số hạng phạt λz^2 cho từng mẫu. Đạo hàm của số hạng phạt này theo w_{11} là gì?

A $2\lambda z\, v_1 f'(z_1)\, x_1$

B $2\lambda z\, x_1$

C $2\lambda z\, v_1 x_1$

D $2\lambda z\, f'(z)\, v_1 f'(z_1)\, x_1$

PHỎNG THEO MIT 6.036 · Final, Spring 2022 · Q2(a)

CÂU 7

Loss MSE lấy trung bình trên B mẫu và c đầu ra, như mặc định của `nn.MSELoss`. Đạo hàm theo mỗi phần tử của P bằng phần dư nhân với hệ số nào?

A $2/B$

B $2c/B$

C $2/(Bc)$

D $1/(Bc)$

CÂU 8

Hai mẫu cùng dùng một trọng số w. Đạo hàm của loss từng mẫu theo w là 6 và -2. Loss của batch là trung bình hai loss. Gradient theo w bằng bao nhiêu?

A 4

B -12

C -2

D 2

CÂU 9

Forward đã dùng `W2` cũ. Trong backward tự viết, code cập nhật `W2` trước rồi mới tính `G1 = G2 @ W2.T`. Sửa lỗi này thế nào?

A Tính hết mọi gradient từ lần forward cũ, rồi mới cập nhật

B Cập nhật `W1` trước, để hai lớp cùng dùng trọng số mới

C Chia learning rate của lớp đầu cho số lớp để bù sai lệch

D Sau khi cập nhật `W2`, tính lại loss từ output cũ

CÂU 10

Mạng một đầu vào, hai neuron ẩn sigmoid, đầu ra linear: $\hat{y} = v_1\sigma(w_1x + c_1) + v_2\sigma(w_2x + c_2) + d$. Khởi tạo hai neuron giống hệt nhau, tức $w_1 = w_2$, $c_1 = c_2$, $v_1 = v_2$, rồi chạy batch gradient descent với MSE. Chuyện gì xảy ra?

A Trọng số đứng yên, vì gradient bằng 0 ngay từ đầu

B Hai neuron mãi giống nhau, mạng không hơn một neuron

C Hai neuron tự tách nhau sau vài bước, nhờ nhiễu làm tròn

D Neuron thứ hai chết dần, chỉ neuron thứ nhất còn học

PHỎNG THEO MIT 6.036 · Final, Spring 2022 · Q4(h)

Đọc thêm

- D2L — Backpropagation: theo dõi các biến trung gian và quy tắc chuỗi trong mạng nhiều lớp.
- Goodfellow, Bengio, Courville — Deep Learning, chương 6: đọc về feedforward network, khả năng biểu diễn và graph tính toán.
- PyTorch 2.9 — Linear: kiểm quy ước weight và shape trước khi chuyển mô hình tự cài sang thư viện.